

コンテキストエンジニアリング

LLMの特性理解・指示プロンプト設計・AIエージェントワークフロー

アジェンダ

1. **対話型LLMの工夫と注意点** - LLMの基本特性を理解する
2. **Reasoningモデルの進化** - 思考過程の出力と性能向上
3. **指示プロンプト開発の基礎** - 全体指針と記法
4. **指示プロンプトの基本構造** - 構成要素の整理
5. **指示プロンプトの精度向上技法** - Few-Shot / CoT
6. **AIエージェントの登場と課題** - なぜエージェントが注目されたか
7. **ワークフロー化によるコンテキスト分散** - 精度向上のための設計
8. **エージェントワークフロー設計** - 具体例で学ぶ設計の勘所
9. **コンテキスト肥大化への対策** - Skill / MCP / コスト管理

Part 1

対話型LLMに施された工夫と注意点

ロール指定による振る舞いの切り替え

- LLMのメカニズムは**テキストの続きを予測する機能**
- チャットアプリでは立場を切り替えて対話のようにトークンを生成する
- 現代LLMでは**特殊トークン**でどの立場の入力かを判断（ロールの付与）

LLMに指定できるロールの例

システム / 開発者 / ユーザー / アシスタント / ツール

応答例への追従性（Few-Shot Prompting）

- 入力と出力の例をコンテキストに付与する
- LLMはその**回答形式を参考にして**出力する

入力例: "東京の天気は?"

出力例: {"city": "東京", "weather": "晴れ", "temp": 25}

入力: "大阪の天気は?"

→ LLMは同じJSON形式で応答する

- これを **Few-Shot Prompting** と呼ぶ

入力テキストの位置による認識精度の差

Lost in the Middle

LLMはコンテキスト内の情報に対し、位置によって注意の度合いが異なる

認識精度： 末尾 > 冒頭 > 中盤

対策

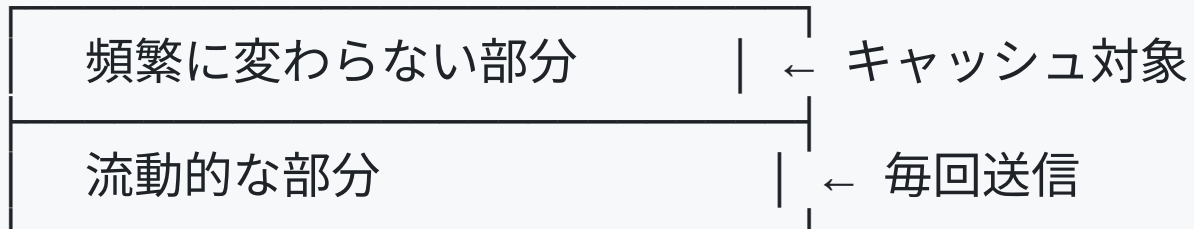
- 重要な情報は常に **末尾か冒頭** に置く
- コンテキストの量を **増やしすぎない**

過去会話の揮発性

- LLMは基本的に、推論リクエスト間で**会話履歴を保持しない揮発性の計算機**

コンテキストキャッシュによる対策

- 不変部分を再入力せず**高速・低コスト化**
- キャッシュ境界を意識する



→ この2つを**なるべく分離**することが重要

言語間の性能格差

- 自然言語、プログラミング言語により**性能に差がある**
- 英語での学習データが最も多い → 英語が最も高精度
- 日本語は英語に比べて精度が落ちる傾向

実務での対策

- 制約・禁止事項は英語で書く
- JSON/XMLのキー名・タグ名は英語
- 条件分岐・ロジック記述は英語が有効

Part 2

Reasoningモデルの進化

Reasoningモデルの登場経緯

- 回答を出すまでに**思考過程を出力**しながら、コンテキストを充実させつつ論理を磨く
- **Chain of Thought (CoT)** の考え方がベース

従来モデルとの違い

従来: 入力 → 出力

Reasoning: 入力 → 思考1 → 思考2 → ... → 出力

思考過程がコンテキストに蓄積されることで、**論理の一貫性**が向上

Reasoningモデル特有の挙動

長所

- ロングコンテキストの解釈性が高い
- 複雑な推論タスクでの精度向上

短所

- 最終出力までの待機時間が長い
- 思考中のツール使用に制約がある場合がある

→ タスクの性質に応じてReasoningモデルと通常モデルを使い分ける

プロンプトとコンテキストの概念の変遷

時代	内容
初期	入力文章から続きを出力するだけ
現代	振る舞いの指示、ツール定義、入出力例、ツール取得結果、会話履歴、出力形式定義、ユーザー入力

→ コンテキスト全体の設計 = コンテキストエンジニアリング

Part 3

指示プロンプト開発の基礎

前提: リファレンスの活用

- モデルがリリースされるたびに**指示プロンプトのガイド**が公開される
- モデル固有の性質を鑑みた指示内容も公開されている
- 追加でガイドを参照することを推奨

全体指針

指示プロンプト = LLMの挙動制御ならびに挙動設計書

指針1: 曖昧語の排除、具体性・再現性の確保

NG例

なるべくちゃんとした回答をしてください

OK例

回答は3文以内で、根拠となるデータソースを明記してください

ポイント

- 「なるべく」「できるだけ」「ちゃんと」を避ける
- 背景・やってほしいこと・制約・出力形式を**構造化**する
- Markdown記法・箇条書きで**自由記述部分を最小化**

指針2: 英語の使用

以下の要素は**英語で記述**することで精度が向上する:

要素	例
制約・禁止事項	You MUST NOT ...
関数名・変数名	getUserName
IT固有用語	authentication , latency
JSON出力時のキ一名	"status" , "error_code"
XMLタグ	<instruction> , <output>
条件分岐・繰り返し記述	If ... then ...

指針3: 否定形を避ける

理解しにくい

Aを選ばないで

理解しやすい

Aを選ぶことは禁止されています

効果的なテクニック

- 記載位置を**上位**にする
- 冗長性を持たせる: 「A以外を選んでください (Aを選ぶことは禁止)」
- **英語**で記述する

指針4: 指示を矛盾させない

矛盾が生じるケース

- RAGなどの参考情報で古い情報が混入
- ユーザーからの指示をシステムロールに入れてしまう

対策

- LLMにプロンプトを確認させる工程を設けて**事前検知**
- 古い情報が混ざる可能性を明示する
- **日付などのメタデータ**を含める
- ユーザーがコントロールできるテキストは指示プロンプトに入れない

指針5～7: その他の重要指針

5. 複数項目の指示・出力はなるべく避ける

- マルチタスクよりもシングルタスクが得意
- 並列実行で遂行度UP、待ち時間も減少

6. 重要な情報は冒頭に書く

- 指示プロンプトの先頭が最も重要な情報と認識される

7. HowよりもWhatを重視する

- タスクのゴールを細部まで言語化する
- ユーザーからコンテキストを引き出すための指示も重要

Part 4

指示プロンプトの記法

ベースとなる記法: Markdown

- 次に何が書かれるか**簡潔かつ明確**に示せる
- LLMの解釈がしやすく、人間にとっても書きやすい
- 多くのモデルが**Markdown前提**で調整されている

活用する要素

見出し / 箇条書き / 番号付きリスト / 強調 / コードブロック / 引用

注意点

- パートの区切りがわかりにくい場合がある
- 表形式の表現能力がやや低い

その他の記法

XMLライクなタグ

```
<instruction>ユーザーの質問に回答してください</instruction>
```

パートの明確な区切りに有効

JSON / YAML / HTML

出力形式の定義や複雑な構造の表現に利用

→ Markdownをベースに、必要に応じて**XMLタグ**や**JSON**で補完する

Part 5

指示プロンプトの基本構造

構成要素の全体像

- | | |
|--------------------------|----------|
| 1. Role | - 役割定義 |
| 2. Task | - タスク定義 |
| 3. Input | - 入力情報 |
| 4. Output Format | - 出力形式 |
| 5. Guidelines | - ルール・注意 |
| 6. Prohibited Actions | - 禁止事項 |
| 7. Timestamp / User Info | |
| 8. Knowledge Base | - 参照情報 |
| 9. Example | - 例示 |

Role（役割定義）

具体的な情報を入れることで出力品質が向上する:

- **職業名:** ソフトウェアエンジニア、データサイエンティスト
- **役割:** テックリード、コードレビュアー
- **所属:** ○○チーム
- **ポジティブなキーワード:** 「優秀な」「経験豊富な」

あなたは優秀なシニアソフトウェアエンジニアです。
10年以上の実務経験を持ち、コードレビューに精通しています。

Task（タスク定義）

要素	内容
目的・背景	なぜこのタスクを実行するのか
スコープ	タスクの範囲や制約（やってはいけないことを含む）
手順・チェックポイント	具体的なステップや達成すべきこと
完了条件	完了とみなされる条件

→ ゴールを明確にすることでLLMの出力精度が向上

Input / Output Format

Input（入力）

- 入力データに対して**解説を与える**
- 何が含まれ、どう使うべきかを明示

Output Format（出力形式）

- JSON, YAMLなど出力データの**構造を定義**

```
{  
  "summary": "要約テキスト",  
  "category": "カテゴリ名",  
  "confidence": 0.95  
}
```

Guidelines / Prohibited Actions

Guidelines（ルール・諸注意）

- タスクのルールや注意事項を記載
-  入力項目が多すぎるとコスト面やタスク精度に悪影響

Prohibited Actions（禁止事項）

- 明示的に禁止事項を書く（例: 推測で書くことを禁止）
- 管理重要度が高い場合が多い
- すべての項目で禁止である旨を繰り返し書いて強調するのも有効

Timestamp / User Info / Knowledge Base / Example

Timestamp • User Information

- 最新情報の把握、ユーザー情報の聞く手間を省く
- タスク遂行に必要な情報はなるべくコンテキストに取り込む

Knowledge Base

- LLMが常に参照すべき情報を与える
- 問い合わせ頻度の高い情報はあらかじめ入れておくほうが効率的

Example

- 例示はふるまいの制御に非常に有効

Part 6

指示プロンプトの精度向上技法

Few-Shot Prompting

2つのパターン

1. 指示プロンプトの**Example**として与える
2. APIに与える**対話履歴の形式**で例示する

注意点

- 例示へのオーバーフィッティング
 - 与えた例に**過度に追従**してしまうことがある
- もともと学習されていないタスクは、いくら例示しても意味がないことがある

Few-Shot Prompting vs ファインチューニング

観点	Few-Shot Prompting	ファインチューニング
方法	少量の例をコンテキストに含める	モデルの再学習が必要
コスト	推論時のトークン消費のみ	学習データ準備 + 学習 + コンピューティング費用
専門知識	不要	必要
能力拡張	既存能力の範囲内	本来対応できないタスクも可能に

その他の精度向上手法

プロンプト改善・RAGより後に検討する手法:

- **DPO (Direct Preference Optimization)** : 複数回答の中から良い方を選ぶ
- **RFT (Reinforcement Fine-tuning)** : Reasoningモデルの思考過程を改善する

Chain of Thought (CoT)

思考過程を出力させることでLLMの論理的推論能力を高める手法

「解法の探索空間を狭め、誤りの混入を減らす」ことで性能向上

CoTの応用

- **論理の構成:** ステップバイステップで推論
- **LLM内部知識の活用:** 思考中に関連知識を引き出す
- **再帰修正:** 自身の出力を見直して修正
- **指示のRecall:** 長い指示を思考中に再確認
- **Structured Output + CoT:** 構造化出力と組み合わせて実装

Part 7

AIエージェントはなぜ注目されたのか

チャットUIのLLMアプリケーションの問題点

- 1. 自分からは動かない - いわゆる指示待ち
- 2. Capabilityの不明瞭さ - 何ができるかわかりにくい
- 3. プロンプト作りのハードルの高さ - ユーザーに負担

市場が期待したAIエージェント像

期待	内容
目標思考	ゴールに向かって自律的に動く
自律性	人間の介入なしにタスクを進める
ツール利用	外部ツールを適切に選択・実行
自己評価・修正	結果を評価し、必要に応じて修正

プロンプトベースエージェント

最も簡単なAIエージェント

- Reasoningモデル + ツールで動作
- Responses APIなどを通じてWeb検索やコード実行（組み込みツール）
- MCP や Function Calling を介した外部ツール連携
- **指示プロンプトを与えるだけで十分なケースも増えている**

課題

- 計画立案と実行がAPIの中で進む
- 複数ステップのタスク状態を**明示的に管理しにくい**

プロンプトベースエージェントの限界

精度低下の要因

- ツール選択の精度低下 - 多すぎるツール定義
- 手順の誤り - ステップを飛ばす・間違える
- 制約の無視 - コンテキストが長くなると守らなくなる
- 非機能面の問題 - コスト・レイテンシの増大

根本対策

1. プロンプトの改善
2. 宣言型ワークフロー化によるタスク分割
3. AIエージェントの独自開発

Part 8

ワークフロー化によるコンテキストの分散

ワークフロー化の基本思想

タスクの流れを**複数のステップに分割**し、それぞれを独立したAIエージェントやシステムで処理する

→ AIエージェント1つあたりの**コンテキストを削減**し、精度を向上させる



ワークフローの構成要素

1. ルールベース処理

- AIを使う必要がない処理は、ルールベースのほうが**早く正確**

2. シンプルなAI/ML処理

- 「専用の小さな入出力」に閉じ込める
- エージェントが保持するコンテキストを**削減**

3. AIEージェント

- ユーザーやツールの応答次第で**柔軟に行動を変えたい**ケースで用いる

AIエージェント分割判断の7つの観点（1/2）

観点	効果
順序制約の強さ	ワークフロー側で順序を矯正し手順誤りを構造的に防止
コンテキストの独立性	工程間の干渉が減り精度が安定（局所最適化もしやすい）
コンテキスト量の増減	要約・抽出・正規化を専用ステップ化しコスト削減と精度安定を両立
ツール境界	ツール定義を減らしツール選択ミスとコンテキスト肥大化を抑制

AIエージェント分割判断の7つの観点（2/2）

観点	効果
失敗時の切り戻し単位	失敗したステップだけ再実行しコストとレイテンシ増大を抑制
品質評価のしやすさ	原因切り分けが容易になりどこで品質が崩れたか特定しやすい
権限秘匿境界	アクセス制御・監査・マスキングを工程単位で設計し漏洩リスク低減

ワークフロー化の注意点

AIエージェントが汎用かつ自律的にタスクを完遂できれば**本来必要ない対応**

推奨アプローチ

1. まずは**単一プロンプトベースエージェント**で試す
 2. いけるところまで**簡易な実装**で進める
 3. どうしても精度が出ないとき、**一部だけワークフローに切り出す**
- 過度な分割は複雑性を増すだけ。**必要最小限**のワークフロー化を心がける

市場が期待した「AIエージェント」の正体

- 多くの企業が求めているのは本質的には**作業の自動化**
- 自動化のためにAIエージェントが期待され、バズワード的に広まった
- 動的判断ですべてを完結させることが難しく、**ワークフロー化**が採用された
- 現在ではワークフローまで含めた自動化が広義に**マルチエージェント**と呼ばれる

結論

「ルールベース処理」「シンプルなAI/ML処理」「AIエージェント」を
パーツとしたワークフローシステムの開発が求められている

Part 9

エージェントワークフロー設計の具体例

サンプルシナリオ

コンサルティングファームA社の課題

- 若手社員が担当領域・顧客の国内外経済ニュースを**手動で調査**
- 社内の過去事例と照らし合わせながら**レポート化**して関連社員へ送付
- 重要度が高いが、探索範囲が膨大で**毎週多大な工数**を費やしている

自動化の検討

→ この作業自動化の要求に対して、どのように応えるべきか？

Step 1: 自動化対象フローの整理

作業フローを可視化する際の確認事項:

確認項目	内容
作業順序	分岐やループの有無
データ	インプット・アウトプットとなるデータ
システムとAPI	使用しているシステムとアクセス手段の有無
権限	認証方式やデータ参照権限のコントロール要否

Step 2: UX/UIの設計

UX/UIはコンテキストの取得とLLMに許される**自由度・挙動と直結する**

検討項目

- **起動条件:** 何を契機にエージェントが動くか
- **ユーザー入力の設計:** 明確かつ最小限に
- **待機時間の設計:** 人をとどまらせず完了を後で通知
- **途中の人間介入:** 承認 / 認証 / コンテキスト追加取得
- **作業状況の可視化:** ステータス / ログ / 通知
- **アウトプット後の挙動:** 結果の表示・後続アクション

待機時間とLLM精度の関係

待機時間 ↔ LLMの思考時間 ↔ タスク精度

- 待機時間はLLMの思考時間と連動
- 思考時間が与えられるかどうかは**タスク精度に直結**

UXの工夫

- ワークフロー起動後、**ユーザーを待たせないUI**を設計
- 作業完了を後で通知
- 待ち時間にユーザーが他のことをできるようにする

Step 3: プロンプトベースエージェントのコンテキスト開発

構成要素

- **インプット**: ユーザー入力 + ツールからの取得結果
- **ツール**: 基本的にツールの呼び出しの繰り返しで完結

指示プロンプトのポイント

- プロンプト通りに動くとは限らず、手順を飛ばしたり間違えることがある
- どのPhaseにいるのか**Reasoning中に毎回確認させる**など世話が必要

ツール定義・モデル選択

- ツール定義の粒度と量を最適化
- タスク特性に応じたモデル・API・パラメータの選択

Step 4: ワークフロー化への移行

ワークフローにおける遷移パターン

パターン	説明
並行実行・集約	複数ステップを同時実行し結果を集約
グループチャット	複数エージェント間の協調
ハンドオフ	エージェント間のタスク引き継ぎ
Agents as Tools	エージェントをツールとして呼び出す

タスク終了判定のパターン

判定方法	内容
ターン数制限	最大ターン数で強制終了
人間による完了判断	人間が確認して完了を判断
LLMによる完了判断	LLM自身が完了を判定
スキーマ充足判定	出力が定義されたスキーマを満たすか検証
テストケース実行	テストを実行して合否を判定

各ステップで引き継ぐコンテキスト

- 会話履歴
- 前ステップの出力結果
- タスク仕様・ゴール・現在のタスク状態

Part 10

コンテキスト肥大化に伴う課題と対策

ツール定義の肥大化

使わない大きなコンテキストを持ち続けることは**性能劣化を誘発するリスク**

対策: Tool Search Tool

- 必要なツール定義を動的に検索・ロード

対策: Skillの活用による段階的コンテキストロード

- 必要になったときに初めてコンテキストをロード
- ただしSkill実行が積み上がると**コンテキストが肥大化**する
- 複数Skillを連続で呼び出すケースは注意が必要

MCP / Skill / エージェント分割の比較

方式	コンテキストへの影響	適用ケース
MCP	肥大化リスクが最も高い	外部ツール連携
Skill	メインエージェントにある程度載る	多段で複雑なサブタスク
エージェント分割	コンテキストを完全に分離	比較的重厚なサブタスク

→ エージェント分割は効果が高いが、生成難易度も高い

ターン経過によって肥大化するコンテキスト

肥大化の原因

- 履歴の蓄積 / ツール結果データ / Reasoning過程

対策

- **履歴の順次除外**: 古いメッセージ履歴から順に除外
- **Reasoning過程の除外**: 思考過程を結果から除外
- **Compact機能・要約**: コンテキストを圧縮・要約
- **Programming Tool Calling**: プログラムで直接処理し結果のみ渡す

→ 除外・要約の**対象選別とタイミング**が重要

コンテキストによるコスト増大への対策

コンテキストキャッシュを切らさない工夫

- 同一セッション内: 不変部分をキャッシュ
- 複数ユーザーからの同時利用: 共通部分のキャッシュ共有

その他のコスト対策

対策	効果
Provisioned Throughput	安定したスループットの確保
バッチAPI	非同期処理によるコスト削減

まとめ

コンテキストエンジニアリングの全体像

コンテキストエンジニアリング		
LLMの特性 理解	プロンプト 設計	AIエージェント ワークフロー
・ ロール ・ Lost in the middle ・ 揮発性 ・ Reasoning	・ 構造化 ・ 曖昧語排除 ・ 英語活用 ・ Few-Shot ・ CoT	・ タスク分割 ・ コンテキスト分散 ・ Skill/MCP/ エージェント分割 ・ コスト管理

LLMの特性を理解し、適切なコンテキストを設計することが、AI活用の品質を決める